Mathematica / Wolfram Language

Task	Command	Notes
Version manager	—	installed with Wolfram product
New project	—	notebook or `.wl` script
Add dependency	`PacletInstall["name"]`	Wolfram paclet system
Install deps	`Needs["pkg`"]`	load package
Build	—	interpreted language
Run	`wolframscript -file s.wl`	CLI execution
REPL	`wolframscript`	or notebook interface
Test	`VerificationTest[expr, expected]`	built-in test framework
Format	—	notebook auto-formats
Lint	`CodeInspector`	`PacletInstall["CodeInspector"]`
LSP	—	notebook is primary IDE

Basic Syntax

`f[x, y]`	Function call
`x = expr`	Assignment
`x := expr`	Delayed assignment
`f[x_] := body`	Function definition
`x /. rule`	Apply rule
`x //. rule`	Apply repeatedly
`expr // f`	Postfix apply
`f @ x`	Prefix apply
`f @@ list`	Apply to list
`f @@@ list`	Apply at level 1
`x // f // g`	Pipeline
`(* comment *)`	Comment

Patterns & Rules

`_`	Any expression
`x_`	Named pattern
`_Integer`	Head pattern
`x_?NumericQ`	Conditional pattern
`__` / `___`	One+ / zero+ args
`x_ /; cond`	Pattern with guard
`a -> b`	Rule
`a :> b`	Delayed rule
`Cases[xs, pat]`	Extract matches
`MatchQ[expr, pat]`	Test pattern
`Replace[e, rules]`	Apply rules

Lists

`{a, b, c}`	List
`xs[[i]]`	Element (1-based)
`xs[[i;;j]]`	Span / slice
`xs[[-1]]`	Last element
`Length[xs]`	Length
`Join[a, b]`	Concatenate
`Append[xs, x]`	Append
`Prepend[xs, x]`	Prepend
`Flatten[xs]`	Flatten nested
`Partition[xs, n]`	Chunk by n
`Range[a, b, step]`	Number range
`Table[f[i], {i, n}]`	Generate list

Functional

`Map[f, xs]`	Apply f to each
`f /@ xs`	Map (operator)
`Select[xs, p]`	Filter
`Fold[f, init, xs]`	Left fold
`FoldList[f, init, xs]`	Scan / accumulate
`MapThread[f, {a, b}]`	Zip + map
`Thread[f[a, b]]`	Thread over lists
`Nest[f, x, n]`	Apply f n times
`NestList[f, x, n]`	Collect iterations
`FixedPoint[f, x]`	Iterate to fixpoint
`Function[{x}, body]`	Named pure fn
`#1, #2`	Slot (arg 1, 2)
`f[#] &`	Pure function
`Composition[f, g]`	$f \circ g$
`Identity`	Identity function

Control Flow

`If[cond, then, else]`	Conditional
`Which[c1,v1, c2,v2]`	Multi-branch
`Switch[x, p1,v1, ...]`	Pattern switch
`Do[body, {i, n}]`	For loop
`While[cond, body]`	While loop
`Module[{x=v}, body]`	Local scope
`Block[{x}, body]`	Dynamic scope
`With[{x=v}, body]`	Constants

Associations (Dicts)

`<|"a"->1, "b"->2|>`	Association
`assoc["key"]`	Lookup
`Keys[a] / Values[a]`	Keys / values
`Merge[a, b]`	Merge
`KeySelect[a, p]`	Filter by key
`AssociationMap[f, a]`	Map over assoc

String Operations

`StringJoin[a, b]` / `a <> b`	Concatenate
`StringSplit[s, ","]`	Split
`StringReplace[s, rule]`	Replace
`StringMatchQ[s, pat]`	Pattern test
`StringCases[s, pat]`	Extract matches
`ToString[expr]`	Convert to string
`StringTemplate["<*x*>"]`	Template

Debugging

`?f`	Show definition
`??f`	Full information
`Head[expr]`	Expression head
`FullForm[expr]`	Internal form
`TreeForm[expr]`	Tree visualization
`Trace[expr]`	Evaluation trace
`Echo[expr]`	Print and return
`AbortProtect[...]`	Protect from abort

Mathematica — Mathematics & Data

Calculus

`D[f, x]`	Derivative
`D[f, {x, n}]`	n-th derivative
`D[f, x, y]`	Partial derivatives
`Integrate[f, x]`	Indefinite integral
`Integrate[f, {x, a, b}]`	Definite integral
`NIntegrate[f, {x,a,b}]`	Numerical integral
`Limit[f, x -> a]`	Limit
`Series[f, {x, a, n}]`	Taylor series
`Normal[%]`	Series to polynomial
`DSolve[ode, y, x]`	Solve ODE
`NDSolve[ode, y, {x,a,b}]`	Numerical ODE
`Sum[f, {i, a, b}]`	Summation
`Product[f, {i, a, b}]`	Product

Algebra

`Solve[eqn, x]`	Solve equation
`NSolve[eqn, x]`	Numerical solve
`Reduce[expr, x]`	Reduce conditions
`Simplify[expr]`	Simplify
`FullSimplify[expr]`	Aggressive simplify
`Expand[expr]`	Expand
`Factor[expr]`	Factor
`Apart[expr]`	Partial fractions
`Together[expr]`	Common denominator
`Collect[expr, x]`	Collect terms

Linear Algebra

`m . v`	Matrix multiply
`Inverse[m]`	Matrix inverse
`Det[m]`	Determinant
`Transpose[m]`	Transpose
`Eigenvalues[m]`	Eigenvalues
`Eigenvectors[m]`	Eigenvectors
`SingularValueDecomp...[m]`	SVD
`LinearSolve[m, b]`	Solve mx = b
`MatrixRank[m]`	Rank
`IdentityMatrix[n]`	Identity
`DiagonalMatrix[v]`	Diagonal
`Norm[v]`	Vector norm

Probability & Statistics

`Mean[xs]`	Mean
`Median[xs]`	Median
`StandardDeviation[xs]`	Std dev
`Variance[xs]`	Variance
`Correlation[a, b]`	Correlation
`NormalDistribution[`μ`,`σ`]`	Normal dist
`PDF[dist, x]`	Probability density
`CDF[dist, x]`	Cumulative dist
`RandomVariate[dist, n]`	Random samples
`Probability[event, dist]`	Exact probability
`Expectation[f, dist]`	Expected value
`LinearModelFit[data, ...]`	Linear regression

Plotting

`Plot[f, {x, a, b}]`	2D function plot
`ListPlot[data]`	Scatter plot
`ListLinePlot[data]`	Line plot
`Plot3D[f, {x,..}, {y,..}]`	3D surface
`ContourPlot[f, ...]`	Contour plot
`ParametricPlot[...]`	Parametric curve
`Histogram[data]`	Histogram
`Show[p1, p2]`	Overlay plots
`GraphicsRow[{...}]`	Side by side
`Export["f.pdf", plot]`	Export figure
`Manipulate[plot, {p,..}]`	Interactive slider

Data Import / Export

`Import["f.csv"]`	Import file
`Import["f.csv", "Dataset"]`	As Dataset
`Export["f.csv", data]`	Export to file
`Import[url]`	Import from URL
`SemanticImport["f.csv"]`	Smart import

Dataset (Structured Data)

`Dataset[data]`	Create Dataset
`ds[All, "col"]`	Select column
`ds[Select[p], All]`	Filter rows
`ds[GroupBy["k"], Mean]`	Group + aggregate
`ds[SortBy["col"]]`	Sort
`ds[Total, "col"]`	Sum column
`Normal[ds]`	To raw data

Number Theory

`PrimeQ[n]`	Primality test
`FactorInteger[n]`	Prime factors
`GCD[a, b]`	GCD
`LCM[a, b]`	LCM
`Mod[a, m]`	Modular arith
`PowerMod[a, b, m]`	Modular power
`Prime[n]`	n-th prime
`Fibonacci[n]`	Fibonacci number

Special Functions

`Gamma[z]`	Gamma function
`BesselJ[n, z]`	Bessel function
`LegendreP[n, x]`	Legendre polynomial
`SphericalHarmonicY`	Spherical harmonics
`EllipticK[m]`	Elliptic integral
`HypergeometricPFQ`	Hypergeometric

Constants

`Pi` (π)	3.14159…
`E` (e)	2.71828…
`I` (i)	Imaginary unit
`Infinity` (∞)	Infinity
`EulerGamma`	Euler–Mascheroni
`GoldenRatio`	ϕ = 1.61803…
`N[expr, digits]`	Numerical value